COURSE PROJECT DOCUMENTATION

GameRig

Digital Game Library Management System

Project & Academic Specification

Course Title:	Advanced Database Management System (ADBMS)
Database Engine:	MySQL 8.0+ (InnoDB Engine)
Architecture:	Relational Model (6 Core Entities, BCNF)
Full Stack Stack:	React 18 + Vite, Node.js + Express, MySQL2
Integrity Models:	ACID Transactions, Row Locks, Stored Routines
Date:	Academic Term 2025–2026

Comprehensive Database Design, Query Execution, Index Optimization & Concurrency Analysis

Table of Contents

Systematic Project Index & Page References

S. No.	Topics	Page No.
1	**Introduction**	3
1.1	Objectives	3
1.2	Project Description	3
2	**Phase I Database Design**	5
2.1	Identification of Entities and Attributes	5
2.2	ER Diagram	6
2.3	Relational Schema	7
2.4	Normalization	8
3	**Phase II Database Creation and Queries**	9
3.1	Creating Tables Using SQL	9
3.2	Inserting and Updating Records	12
3.3	SQL Queries	13
3.4	Aggregate Functions	15
4	**Phase III Data Querying Join and Views**	17
4.1	Joins	17
4.2	Subqueries	19
4.3	Views	21
4.4	Functions and Procedures	23
5	**Phase IV Transactions and Data Integrity**	25
5.1	COMMIT, ROLLBACK and SAVEPOINT	25
5.2	Integrity Constraints	26
5.3	Concurrency Issues	27
7	**Conclusion**	30
8	**References**	30
9	**Appendix**	31

1. Introduction

1.1 Objectives

The primary objective of the **GameRig** project is to design, model, implement, and validate an enterprise-grade digital gaming library and commerce database system for modern digital video game distribution. In modern digital software marketplaces (comparable to Steam, Epic Games, or PlayStation Store), maintaining strict transaction atomicity, real-time catalog search performance, user ownership tracking, review credibility, and referential data integrity across multi-tiered entities is paramount.

Specific academic and practical objectives include:

- **Relational Data Modeling:** Design an optimal schema in Boyce-Codd Normal Form (BCNF) eliminating insertion, update, and deletion anomalies.
- **Transactional Atomicity (ACID):** Guarantee that multi-table operations (e.g., deducting payment and provisioning game licenses to player libraries) execute under all-or-nothing transactional guarantees.
- **Concurrency & Integrity Controls:** Implement pessimistic row-level locking (`SELECT ... FOR UPDATE`) and comprehensive foreign key cascade rules to prevent race conditions, orphan records, and overselling.
- **Analytical Query Optimization:** Utilize non-clustered B-Tree indexes, composite views, nested subqueries, and stored routines to minimize disk I/O and optimize query execution plans.
- **Full-Stack Interoperability:** Connect the MySQL database to an Express.js REST API with connection pooling and a responsive React frontend interface.

1.2 Project Description

GameRig is a full-featured digital game store and personal library management portal. The system models a two-sided platform where administrators manage game listings, prices, and system users, while players register accounts, browse games, submit verified star ratings and text reviews, purchase digital licenses, and organize their acquired software collections.

The relational architecture consists of six dedicated entities:

1. `ADMIN` : Manages credentials, roles, and administrative authority over game catalog releases.
2. `USER` : Represents consumer player accounts storing profile identities, unique emails, and registration dates.
3. `GAME` : Catalogs titles, descriptions, genres, platform compatibility, release schedules, pricing, and administering managers.
4. `REVIEW` : Records player evaluations, numerical 1–5 star scores, qualitative feedback, and posting timestamps with cascade-on-delete protections.
5. `PAYMENT` : Stores financial transaction ledgers including payment channels (UPI, Credit/Debit Card, Net Banking), amounts, and audit timestamps.
6. `LIBRARY` : Acts as the definitive entitlement ledger, binding a specific user to an owned game with a composite unique constraint preventing duplicate ownership purchases.

Core DBMS Architecture Highlights

GameRig implements MySQL's InnoDB storage engine to provide strict ACID guarantees, multi-version concurrency control (MVCC), foreign key referential integrity with cascading updates, and deadlock avoidance mechanisms.

2. Phase I Database Design

2.1 Identification of Entities and Attributes

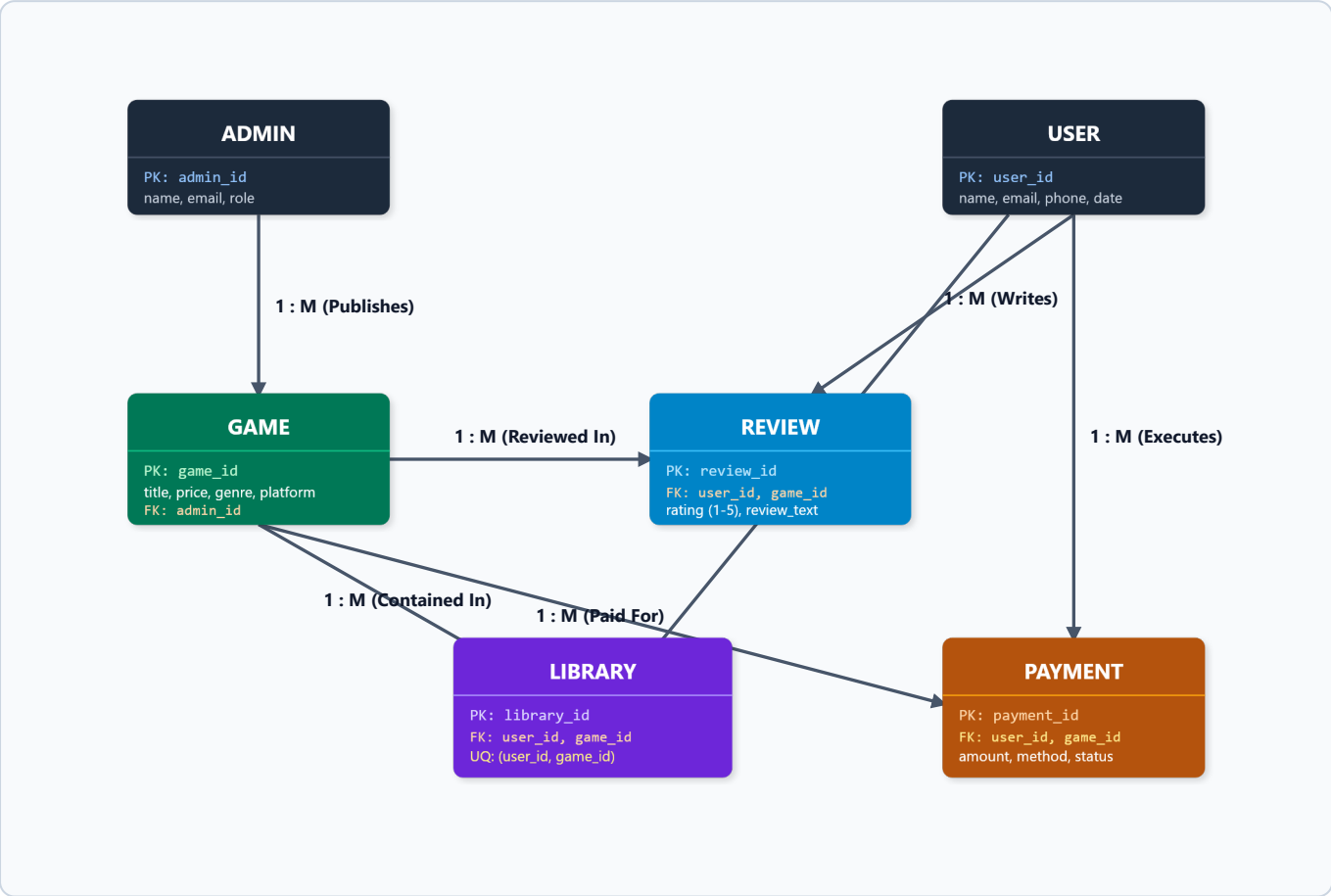
During the conceptual design phase, six distinct entities were identified to represent all functional requirements without data redundancy.

Entity Name	Attributes	Primary Key	Foreign Keys & Dependencies
ADMIN	`admin_id`, `name`, `email`, `password`, `role`	`admin_id`	None (Independent Entity)
USER	`user_id`, `name`, `email`, `password`, `phone`, `date_joined`	`user_id`	None (Independent Entity; `email` is UNIQUE)
GAME	`game_id`, `title`, `description`, `genre`, `price`, `platform`, `release_date`, `image`, `admin_id`	`game_id`	`admin_id` → references `ADMIN(admin_id)`
REVIEW	`review_id`, `user_id`, `game_id`, `rating`, `review_text`, `review_date`	`review_id`	`user_id` → `USER(user_id)` `game_id` → `GAME(game_id)`
PAYMENT	`payment_id`, `user_id`, `game_id`, `amount`, `payment_status`, `payment_method`, `payment_date`	`payment_id`	`user_id` → `USER(user_id)` `game_id` → `GAME(game_id)`

LIBRARY	`library_id`, `user_id`, `game_id`, `purchase_date`, `status`	`library_id`	`user_id` → `USER(user_id)`
			`game_id` → `GAME(game_id)`
			`UNIQUE(user_id, game_id)`

2.2 ER Diagram

The Entity-Relationship (ER) diagram below models the conceptual database architecture, cardinalities, associative entities, and relationship sets across the GameRig system.



2.3 Relational Schema

The relational schema formally maps conceptual entities and relationships into normalized relations. Primary keys are underlined (**PK**), and foreign keys are designated with an arrow (→).

ADMIN (admin_id, name, email, password, role)
- AK: (email) [Unique Alternate Key]

USER (user_id, name, email, password, phone, date_joined)
- AK: (email) [Unique Alternate Key]

GAME (game_id, title, description, genre, price, platform, release_date, image, admin_id)
- FK: admin_id → ADMIN(admin_id) ON DELETE SET NULL

REVIEW (review_id, user_id, game_id, rating, review_text, review_date)
- FK: user_id → USER(user_id) ON DELETE CASCADE
- FK: game_id → GAME(game_id) ON DELETE CASCADE

PAYMENT (payment_id, user_id, game_id, amount, payment_status, payment_method, payment_date)
- FK: user_id → USER(user_id) ON DELETE CASCADE
- FK: game_id → GAME(game_id) ON DELETE CASCADE

LIBRARY (library_id, user_id, game_id, purchase_date, status)
- FK: user_id → USER(user_id) ON DELETE CASCADE
- FK: game_id → GAME(game_id) ON DELETE CASCADE
- UQ: (user_id, game_id) [Prevent multiple concurrent ownership of same license]

2.4 Normalization

Database normalization is the formal mathematical technique used to organize relations, minimize redundancy, and prevent update, insertion, and deletion anomalies. GameRig was engineered step-by-step from Unnormalized Form (UNF) through to Boyce-Codd Normal Form (BCNF).

1. First Normal Form (1NF)

Rule: Each relation must contain only atomic (indivisible) attribute values; no repeating groups or multivalued attributes are permitted.

- **Violation in UNF:** If a single user record stored multiple purchased games as a comma-separated array (e.g. `games_owned = "Cyberpunk 2077, Elden Ring"`), querying specific games requires parsing string tokens and prevents foreign key index lookup.
- **1NF Transformation:** Repeating collections were dismantled. An associative table (`LIBRARY`) was introduced where every individual ownership instance is stored as a single, atomic row.

2. Second Normal Form (2NF)

Rule: Relation must be in 1NF and possess *no partial functional dependencies* (all non-prime attributes must be fully functionally dependent on the entire primary key).

- **Analysis:** In composite-key candidate tables (such as `(user_id, game_id)`), storing the game's price or description would create a partial dependency, because `price` depends solely on `game_id`, not `user_id`.
- **2NF Transformation:** The `GAME` relation was cleanly separated so that `title, description, price, genre` depend solely on the single candidate key `game_id`. In `LIBRARY`, surrogate key `library_id` is the primary key, and non-prime attributes (`purchase_date, status`) depend fully on the complete key.

3. Third Normal Form (3NF)

Rule: Relation must be in 2NF and contain *no transitive functional dependencies* (for any non-trivial functional dependency $X \rightarrow Y$, X must be a superkey or Y is a prime attribute).

- **Analysis:** If `PAYMENT` stored customer billing address and user name (where $payment_id \rightarrow user_id$ and $user_id \rightarrow user_address$), an update to a user's address would require updating multiple payment rows.
- **3NF Transformation:** Transitive attributes (user identity details) remain strictly in the `USER` table. `PAYMENT` references only `user_id` and records transaction-specific data (`amount, payment_status, payment_method`).

4. Boyce-Codd Normal Form (BCNF)

Rule: For every functional dependency $X \rightarrow Y$, X must be a superkey.

- In all six tables (`ADMIN` , `USER` , `GAME` , `REVIEW` , `PAYMENT` , `LIBRARY`), the determinants of every functional dependency are candidate keys (e.g., `user_id` or `email` in `USER` ; `(user_id, game_id)` in `LIBRARY`). Hence, the entire GameRig schema satisfies BCNF.

3. Phase II Database Creation and Queries

3.1 Creating Tables Using SQL

The following Data Definition Language (DDL) statements initialize the `GameRig` relational database, declare primary keys, configure check constraints, and set up cascading referential integrity.

```sql
CREATE DATABASE IF NOT EXISTS GameRig;
USE GameRig;

-- 1. ADMIN TABLE
CREATE TABLE ADMIN (
    admin_id    INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE,
    password    VARCHAR(255) NOT NULL,
    role        VARCHAR(50)  DEFAULT 'Admin'
);

-- 2. USER TABLE (`USER` is a MySQL keyword -> backticks required)
CREATE TABLE `USER` (
    user_id     INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE,
    password    VARCHAR(255) NOT NULL,
    phone       VARCHAR(15),
    date_joined DATE NOT NULL
);

-- 3. GAME TABLE (with CHECK constraint on price)
CREATE TABLE GAME (
    game_id      INT PRIMARY KEY AUTO_INCREMENT,
    title        VARCHAR(150) NOT NULL,
    description  TEXT,
    genre        VARCHAR(50)  NOT NULL,
    price        DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (price >= 0),
    platform     VARCHAR(50)  NOT NULL,
    release_date DATE,
    image        VARCHAR(500),
    admin_id     INT,
    FOREIGN KEY (admin_id) REFERENCES ADMIN(admin_id) ON DELETE SET NULL
);

-- 4. REVIEW TABLE (with CHECK constraint for 1-5 star bounds)
CREATE TABLE REVIEW (
    review_id    INT PRIMARY KEY AUTO_INCREMENT,
    user_id      INT NOT NULL,
    game_id      INT NOT NULL,
    rating       INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    review_text  TEXT,
    review_date  DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES `USER`(user_id) ON DELETE CASCADE,
    FOREIGN KEY (game_id) REFERENCES GAME(game_id)   ON DELETE CASCADE
);
```

```sql
-- 5. PAYMENT TABLE (financial ledger with CHECK amount >= 0)
CREATE TABLE PAYMENT (
    payment_id      INT PRIMARY KEY AUTO_INCREMENT,
    user_id         INT NOT NULL,
    game_id         INT NOT NULL,
    amount          DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
    payment_status  VARCHAR(30)   NOT NULL DEFAULT 'Completed',
    payment_method  VARCHAR(50)   NOT NULL,
    payment_date    DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES `USER`(user_id) ON DELETE CASCADE,
    FOREIGN KEY (game_id) REFERENCES GAME(game_id)   ON DELETE CASCADE
);

-- 6. LIBRARY TABLE (Ownership mapping with Composite UNIQUE constraint)
CREATE TABLE `LIBRARY` (
    library_id     INT PRIMARY KEY AUTO_INCREMENT,
    user_id        INT NOT NULL,
    game_id        INT NOT NULL,
    purchase_date  DATE NOT NULL,
    status         VARCHAR(30) DEFAULT 'Owned',
    FOREIGN KEY (user_id) REFERENCES `USER`(user_id) ON DELETE CASCADE,
    FOREIGN KEY (game_id) REFERENCES GAME(game_id)   ON DELETE CASCADE,
    UNIQUE KEY unique_user_game (user_id, game_id)
);
```

3.2 Inserting and Updating Records

The following DML statements demonstrate inserting sample records, modifying entity attributes through `UPDATE`, and executing constraint-safe deletions.

1. Sample Insertions (DML)

```
-- Insert System Admin & Moderator
INSERT INTO ADMIN (name, email, password, role) VALUES
('System Admin', 'admin@gamerig.com', '$2b$10$hashed_pw_here...', 'Super Admin'),
('Alex Mercer',  'alex@gamerig.com',  '$2b$10$hashed_pw_here...', 'Game Moderator');

-- Insert Registered Users
INSERT INTO `USER` (name, email, password, phone, date_joined) VALUES
('John Doe',     'john@gamerig.com',  '$2b$10$hash...', '+19876543210', '2025-01-15'),
('Sarah Connor', 'sarah@gamerig.com', '$2b$10$hash...', '+19876543211', '2025-02-01'),
('Gamer Pro',    'gamer@gamerig.com', '$2b$10$hash...', NULL,          '2025-03-10');

-- Insert Sample Catalog Game
INSERT INTO GAME (title, description, genre, price, platform, release_date, admin_id) VALUES
('Cyberpunk 2077: Phantom Liberty', 'Action RPG in Night City', 'RPG', 59.99, 'PC / PS5', '2023-
09-26', 1),
('Elden Ring: Shadow of the Erdtree', 'Dark fantasy action RPG', 'Action RPG', 39.99, 'PC /
Console', '2024-06-21', 1);
```

2. Record Modifications (UPDATE)

```
-- Scenario: Price discount applied to Grand Theft Auto V
UPDATE GAME
SET    price = 19.99
WHERE  game_id = 4;

-- Scenario: Payment gateway webhook triggers payment state transition
UPDATE PAYMENT
SET    payment_status = 'Completed'
WHERE  payment_id = 5;
```

3. Safe Deletions (DELETE)

```
-- Safely remove unneeded or flagged review (no dependents reference REVIEW)
DELETE FROM REVIEW
WHERE  review_id = 6;
```

3.3 SQL Queries

The following queries showcase filtering with `WHERE`, range evaluations with `BETWEEN`, sorting via `ORDER BY`, and handling `NULL` values.

Query 1: Games filtered by Genre and ordered by Price

```
SELECT game_id, title, genre, price, platform
FROM   GAME
WHERE  genre = 'Action RPG'
ORDER BY price DESC;
```

```
+---------+-----------------------------------+------------+-------+-------------------------+
| game_id | title | genre | price | platform |
+---------+-----------------------------------+------------+-------+-------------------------+
| 6 | Black Myth: Wukong | Action RPG | 59.99 | PC / PS5 |
| 15 | Ghost of Tsushima DIRECTOR'S CUT | Action RPG | 59.99 | PC / PS5 |
| 2 | Elden Ring: Shadow of the Erdtree | Action RPG | 39.99 | PC / PlayStation / Xbox |
+---------+-----------------------------------+------------+-------+-------------------------+
3 rows in set (0.00 sec)
```

Query 2: Mid-Price Range Games ($20.00 to $50.00)

```
SELECT game_id, title, genre, price
FROM   GAME
WHERE  price BETWEEN 20.00 AND 50.00
ORDER BY price ASC;
```

```
+---------+-----------------------------------+--------------------+-------+
| game_id | title | genre | price |
+---------+-----------------------------------+--------------------+-------+
| 4 | Grand Theft Auto V | Action | 29.99 |
| 19 | Palworld | Monster Survival | 29.99 |
| 20 | Minecraft: Java & Bedrock Edition | Sandbox / Survival | 29.99 |
| 2 | Elden Ring: Shadow of the Erdtree | Action RPG | 39.99 |
| 8 | The Witcher 3: Wild Hunt | Open World RPG | 39.99 |
| 16 | Resident Evil 4 Remake | Survival Horror | 39.99 |
| 3 | God of War Ragnarök | Action / Adventure | 49.99 |
+---------+-----------------------------------+--------------------+-------+
7 rows in set (0.00 sec)
```

Query 3: Handling Missing Contact Data (IS NULL / IS NOT NULL)

```
-- Find users without registered phone numbers
SELECT user_id, name, email, phone
FROM   `USER`
WHERE  phone IS NULL;
```

```
+---------+-----------+-------------------+-------+
| user_id | name | email | phone |
+---------+-----------+-------------------+-------+
| 3 | Gamer Pro | gamer@gamerig.com | NULL |
+---------+-----------+-------------------+-------+
1 row in set (0.00 sec)
```

3.4 Aggregate Functions

Aggregate functions perform mathematical summarization over sets of values. In GameRig, aggregates compute average ratings, sales metrics, and player counts using `COUNT`, `AVG`, `SUM`, `GROUP BY`, and `HAVING`.

1. Average Rating and Total Review Volume per Game (GROUP BY & HAVING)

```
SELECT g.game_id, g.title,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       COUNT(r.review_id)      AS total_reviews
FROM   GAME g
LEFT JOIN REVIEW r ON g.game_id = r.game_id
GROUP BY g.game_id, g.title
HAVING COUNT(r.review_id) > 0
ORDER BY avg_rating DESC;
```

```
+---------+----------------------------------+------------+---------------+
| game_id | title | avg_rating | total_reviews |
+---------+----------------------------------+------------+---------------+
| 2 | Elden Ring: Shadow of the Erdtree | 5.00 | 1 |
| 4 | Grand Theft Auto V | 5.00 | 1 |
| 1 | Cyberpunk 2077: Phantom Liberty | 4.00 | 2 |
| 3 | God of War Ragnarök | 4.00 | 1 |
| 5 | Red Dead Redemption 2 | 4.00 | 1 |
+---------+----------------------------------+------------+---------------+
5 rows in set (0.00 sec)
```

2. Total Revenue & Transaction Breakdown per Payment Gateway

```
SELECT payment_method,
       COUNT(*)          AS completed_transactions,
       SUM(amount)       AS gross_revenue,
       ROUND(AVG(amount), 2) AS avg_ticket_size
FROM   PAYMENT
WHERE  payment_status = 'Completed'
GROUP BY payment_method
ORDER BY gross_revenue DESC;
```

```
+----------------+------------------------+---------------+-----------------+
| payment_method | completed_transactions | gross_revenue | avg_ticket_size |
+----------------+------------------------+---------------+-----------------+
| Credit Card | 2 | 119.98 | 59.99 |
| Net Banking | 1 | 49.99 | 49.99 |
| UPI | 1 | 39.99 | 39.99 |
| Debit Card | 1 | 29.99 | 29.99 |
+----------------+------------------------+---------------+-----------------+
4 rows in set (0.00 sec)
```

3. Comprehensive System-Wide Metrics (Single Pass Aggregation)

```
SELECT
    (SELECT COUNT(*) FROM `USER`)                                    AS total_users,
    (SELECT COUNT(*) FROM GAME)                                      AS total_games,
    (SELECT COUNT(*) FROM REVIEW)                                    AS total_reviews,
    (SELECT COUNT(*) FROM `LIBRARY`)                                 AS total_library_records,
    (SELECT ROUND(SUM(amount), 2) FROM PAYMENT WHERE payment_status = 'Completed') AS
total_revenue;
```

```
+-------------+-------------+---------------+----------------------+---------------+
| total_users | total_games | total_reviews | total_library_records | total_revenue |
+-------------+-------------+---------------+----------------------+---------------+
| 3 | 20 | 6 | 5 | 239.95 |
+-------------+-------------+---------------+----------------------+---------------+
1 row in set (0.00 sec)
```

4. Phase III Data Querying Join and Views

4.1 Joins

Relational databases distribute attributes across tables to avoid redundancy. Relational `JOIN` operations recombine related entities into unified analytical projections.

1. Multi-Table Review Audit: USER + REVIEW + GAME (3-Table INNER JOIN)

```
SELECT
    r.review_id,
    u.name AS user_name,
    g.title AS game_title,
    g.genre,
    r.rating,
    r.review_text,
    r.review_date
FROM REVIEW r
JOIN `USER` u ON r.user_id = u.user_id
JOIN GAME g   ON r.game_id = g.game_id
ORDER BY r.review_date DESC;
```

```
+-----------+--------------+-----------------------------------+--------------------+--------+-------
-----------------------------------------------+-------------+
| review_id | user_name | game_title | genre | rating | review_text | review_date |
+-----------+--------------+-----------------------------------+--------------------+--------+-------
-----------------------------------------------+-------------+
| 6 | Gamer Pro | Cyberpunk 2077: Phantom Liberty | RPG | 3 | Good game but a bit overhyped. Still
worth trying. | 2025-04-05 |
| 5 | Sarah Connor | Red Dead Redemption 2 | Open World Action | 4 | One of the greatest narrative
achievements in gaming. | 2025-04-01 |
| 4 | Gamer Pro | Grand Theft Auto V | Action | 5 | Classic game. Online mode still has
replayability. | 2025-03-12 |
| 3 | John Doe | God of War Ragnarök | Action / Adventure | 4 | Graphically stunning and emotionally
rich storyline. | 2025-03-01 |
| 2 | Sarah Connor | Elden Ring: Shadow of the Erdtree | Action RPG | 5 | The hardest and most
rewarding boss fights! 10/10! | 2025-02-15 |
| 1 | John Doe | Cyberpunk 2077: Phantom Liberty | RPG | 5 | Absolute masterpiece after all the
updates! | 2025-02-10 |
+-----------+--------------+-----------------------------------+--------------------+--------+-------
-----------------------------------------------+-------------+
6 rows in set (0.00 sec)
```

2. Customer Entitlement Report: USER + LIBRARY + GAME (INNER JOIN)

```
SELECT
    l.library_id,
    u.name AS owner_name,
    g.title AS game_title,
    g.platform,
    l.purchase_date,
    l.status AS license_status
FROM `LIBRARY` l
JOIN `USER` u ON l.user_id = u.user_id
JOIN GAME g   ON l.game_id = g.game_id
ORDER BY u.name ASC, l.purchase_date DESC;
```

```
+------------+--------------+----------------------------------+-------------------------+----------
-----+----------------+
| library_id | owner_name | game_title | platform | purchase_date | license_status |
+------------+--------------+----------------------------------+-------------------------+----------
-----+----------------+
| 4 | Gamer Pro | Grand Theft Auto V | PC / Console | 2025-03-12 | Owned |
| 3 | John Doe | God of War Ragnarök | PC / PS5 | 2025-03-01 | Owned |
| 1 | John Doe | Cyberpunk 2077: Phantom Liberty | PC / PS5 / Xbox Series | 2025-02-10 | Owned |
| 5 | Sarah Connor | Red Dead Redemption 2 | PC / PS4 / Xbox One | 2025-04-01 | Owned |
| 2 | Sarah Connor | Elden Ring: Shadow of the Erdtree | PC / PlayStation / Xbox | 2025-02-15 | Owned
|
+------------+--------------+----------------------------------+-------------------------+----------
-----+----------------+
5 rows in set (0.00 sec)
```

4.2 Subqueries

Subqueries (nested queries) permit dynamic multi-stage filtering where the result of an inner query serves as the comparison predicate for an outer query.

1. Games with Average Rating Strictly Above the Global Platform Average

```
SELECT
    g.game_id,
    g.title,
    g.genre,
    ROUND(AVG(r.rating), 2) AS game_avg_rating,
    (SELECT ROUND(AVG(rating), 2) FROM REVIEW) AS overall_avg_rating
FROM GAME g
JOIN REVIEW r ON g.game_id = r.game_id
GROUP BY g.game_id, g.title, g.genre
HAVING AVG(r.rating) > (SELECT AVG(rating) FROM REVIEW)
ORDER BY game_avg_rating DESC;
```

```
+---------+-----------------------------------+------------+-----------------+--------------------+
| game_id | title | genre | game_avg_rating | overall_avg_rating |
+---------+-----------------------------------+------------+-----------------+--------------------+
| 2 | Elden Ring: Shadow of the Erdtree | Action RPG | 5.00 | 4.33 |
| 4 | Grand Theft Auto V | Action | 5.00 | 4.33 |
+---------+-----------------------------------+------------+-----------------+--------------------+
2 rows in set (0.00 sec)
```

2. Users Whose Purchase Volume Exceeds the Platform-Wide Average

```
SELECT
    u.user_id,
    u.name,
    u.email,
    COUNT(p.payment_id) AS user_purchase_count
FROM `USER` u
JOIN PAYMENT p ON u.user_id = p.user_id
WHERE p.payment_status = 'Completed'
GROUP BY u.user_id, u.name, u.email
HAVING COUNT(p.payment_id) > (
    SELECT AVG(user_purchases.total_orders)
    FROM (
        SELECT user_id, COUNT(payment_id) AS total_orders
        FROM PAYMENT
        WHERE payment_status = 'Completed'
        GROUP BY user_id
    ) AS user_purchases
)
ORDER BY user_purchase_count DESC;
```

```
+---------+--------------+-------------------+---------------------+
| user_id | name | email | user_purchase_count |
+---------+--------------+-------------------+---------------------+
| 1 | John Doe | john@gamerig.com | 2 |
| 2 | Sarah Connor | sarah@gamerig.com | 2 |
+---------+--------------+-------------------+---------------------+
2 rows in set (0.00 sec)
```

4.3 Views

A database view is a virtual table defined by a stored SQL query. Views provide significant architectural benefits:

- **Security & Privacy:** Masks sensitive fields (such as hashed passwords) from frontend reporting clients.
- **Query Simplification:** Replaces repetitive 4-table join operations with a simple query against a single virtual object.
- **Consistency:** Ensures business logic (e.g. coalescing null payment records) is centralized within the database.

Definition of View: Game_Library_Summary

```sql
DROP VIEW IF EXISTS Game_Library_Summary;

CREATE VIEW Game_Library_Summary AS
SELECT
    u.name AS user_name,
    u.email AS user_email,
    g.title AS game_title,
    g.genre,
    l.purchase_date,
    COALESCE(p.payment_status, 'N/A') AS payment_status,
    l.status AS library_status
FROM `LIBRARY` l
JOIN `USER` u ON l.user_id = u.user_id
JOIN GAME g   ON l.game_id = g.game_id
LEFT JOIN PAYMENT p ON l.user_id = p.user_id AND l.game_id = p.game_id;
```

Executing and Querying the View

```sql
SELECT * FROM Game_Library_Summary;
```

```
+--------------+------------------+----------------------------------+--------------------+--------
-------+----------------+----------------+
| user_name | user_email | game_title | genre | purchase_date | payment_status | library_status |
+--------------+------------------+----------------------------------+--------------------+--------
-------+----------------+----------------+
| John Doe | john@gamerig.com | Cyberpunk 2077: Phantom Liberty | RPG | 2025-02-10 | Completed |
Owned |
| Sarah Connor | sarah@gamerig.com | Elden Ring: Shadow of the Erdtree | Action RPG | 2025-02-15 |
Completed | Owned |
| John Doe | john@gamerig.com | God of War Ragnarök | Action / Adventure | 2025-03-01 | Completed |
Owned |
| Gamer Pro | gamer@gamerig.com | Grand Theft Auto V | Action | 2025-03-12 | Completed | Owned |
| Sarah Connor | sarah@gamerig.com | Red Dead Redemption 2 | Open World Action | 2025-04-01 |
Completed | Owned |
+--------------+------------------+----------------------------------+--------------------+--------
-------+----------------+----------------+
5 rows in set (0.00 sec)
```

4.4 Functions and Procedures

Stored routines encapsulate server-side procedural logic directly on the database engine, reducing network latency and centralizing business enforcement.

1. Stored Function: GetUserGameCount

A deterministic stored function that computes and returns the total number of games registered in a given user's personal game library.

```
DELIMITER $$

CREATE FUNCTION GetUserGameCount(p_user_id INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE game_count INT;
    SELECT COUNT(*) INTO game_count
    FROM `LIBRARY`
    WHERE user_id = p_user_id;
    RETURN game_count;
END$$

DELIMITER ;
```

Testing Function in a SELECT Statement

```
SELECT user_id, name, email, GetUserGameCount(user_id) AS total_games_in_library FROM `USER`;
```

```
+---------+--------------+-------------------+------------------------+
| user_id | name | email | total_games_in_library |
+---------+--------------+-------------------+------------------------+
| 1 | John Doe | john@gamerig.com | 2 |
| 2 | Sarah Connor | sarah@gamerig.com | 2 |
| 3 | Gamer Pro | gamer@gamerig.com | 1 |
+---------+--------------+-------------------+------------------------+
3 rows in set (0.00 sec)
```

2. Stored Procedure: AddGameReview

Accepts user ID, game ID, numerical star rating, and text comments, automatically stamping the review timestamp and returning confirmation metadata.

```
DELIMITER $$

CREATE PROCEDURE AddGameReview(
    IN p_user_id INT,
    IN p_game_id INT,
    IN p_rating INT,
    IN p_review_text TEXT
)
BEGIN
    INSERT INTO REVIEW (user_id, game_id, rating, review_text, review_date)
    VALUES (p_user_id, p_game_id, p_rating, p_review_text, CURDATE());

    SELECT 'Review submitted successfully!' AS status_message,
           LAST_INSERT_ID() AS new_review_id;
END$$

DELIMITER ;
```

5. Phase IV Transactions and Data Integrity

5.1 COMMIT, ROLLBACK and SAVEPOINT

A database transaction is a logical unit of work comprising multiple operations that must satisfy the **ACID** criteria:

- **Atomicity:** All steps succeed together, or the entire transaction is rolled back with zero side effects.
- **Consistency:** The database transitions only between valid states conforming to all schema constraints.
- **Isolation:** Concurrent transactions execute without cross-talk or race conditions.
- **Durability:** Once committed, state changes survive any subsequent server crash or power loss.

1. Successful Transaction Flow with SAVEPOINT

```
START TRANSACTION;

-- Step 1: Concurrency check & Lock user and game rows
SELECT user_id, name FROM `USER` WHERE user_id = 2 FOR UPDATE;
SELECT game_id, title, price FROM GAME WHERE game_id = 3 FOR UPDATE;

-- Step 2: Insert Payment Record
INSERT INTO PAYMENT (user_id, game_id, amount, payment_status, payment_method, payment_date)
VALUES (2, 3, 49.99, 'Completed', 'UPI', CURDATE());

-- Step 3: Establish SAVEPOINT before modifying library
SAVEPOINT BeforeLibraryUpdate;

-- Step 4: Insert Entitlement License in LIBRARY
INSERT INTO `LIBRARY` (user_id, game_id, purchase_date, status)
VALUES (2, 3, CURDATE(), 'Owned');

-- Step 5: Both succeed -> COMMIT atomically
COMMIT;
```

2. Aborted Transaction with Complete ROLLBACK

```
START TRANSACTION;

-- Step 1: User pays $59.99
INSERT INTO PAYMENT (user_id, game_id, amount, payment_status, payment_method, payment_date)
VALUES (1, 1, 59.99, 'Completed', 'Credit Card', CURDATE());

-- Step 2: Library insertion fails because User 1 already owns Game 1!
-- (Triggers Error 1062: Duplicate entry '1-1' for key 'unique_user_game')

-- Step 3: ROLLBACK completely to prevent orphaned payment deduction
ROLLBACK;
```

5.2 Integrity Constraints

Data integrity rules protect the correctness and reliability of the database. The GameRig schema implements three fundamental categories of constraints:

Constraint Category	Implementation in GameRig	System Protection Provided
Entity Integrity	`PRIMARY KEY AUTO_INCREMENT` on all 6 tables	Guarantees every row has a non-null, uniquely identifiable identity.
Referential Integrity	`FOREIGN KEY ... ON DELETE CASCADE / SET NULL`	Prevents orphan records; if a user is deleted, their reviews & payments cascade cleanly.
Domain Integrity	Data types (`DECIMAL(10,2)` , `DATE` , `VARCHAR`)	Restricts column values to valid technical domains and date formats.
Check Constraints	`CHECK (rating BETWEEN 1 AND 5)` , `CHECK (price >= 0)`	Prevents irrational data entry (e.g. negative pricing or 10-star reviews).
Unique Constraints	`UNIQUE(email)` , `UNIQUE(user_id, game_id)`	Blocks duplicate user registrations and prevents buying the same game twice.

Constraint Rejection Verification Tests

```
-- Test 1: CHECK constraint rejection (Rating out of bounds: 6)
INSERT INTO REVIEW (user_id, game_id, rating, review_text, review_date)
VALUES (1, 2, 6, 'Too high!', CURDATE());
-- RESULT: ERROR 3819 (HY000): Check constraint 'review_chk_1' is violated.

-- Test 2: Foreign Key rejection (Invalid User ID 9999)
INSERT INTO PAYMENT (user_id, game_id, amount, payment_status, payment_method, payment_date)
VALUES (9999, 1, 59.99, 'Completed', 'UPI', CURDATE());
-- RESULT: ERROR 1452 (23000): Cannot add or update child row: foreign key constraint fails.

-- Test 3: Composite UNIQUE constraint rejection (Duplicate game purchase)
INSERT INTO `LIBRARY` (user_id, game_id, purchase_date, status)
VALUES (1, 1, CURDATE(), 'Owned');
-- RESULT: ERROR 1062 (23000): Duplicate entry '1-1' for key 'LIBRARY.unique_user_game'.
```

5.3 Concurrency Issues

In multi-user concurrent environments, simultaneous database interactions can cause severe anomalies if unmanaged:

- **Lost Updates:** Two users concurrently purchasing the final available copy, overwriting each other's record state.
- **Dirty Reads:** Reading uncommitted modifications made by another concurrent transaction that subsequently rolls back.
- **Non-Repeatable Reads:** Re-reading a row within a single transaction and receiving modified attribute values.

Production Concurrency-Safe Stored Procedure: ProcessGamePurchase

The following stored procedure handles the full commercial transaction with pessimistic row-level locking (`FOR UPDATE`) and automated error rollback handlers.

```
DELIMITER $$

CREATE PROCEDURE ProcessGamePurchase(
    IN p_user_id INT,
    IN p_game_id INT,
    IN p_amount DECIMAL(10,2),
    IN p_payment_method VARCHAR(50)
)
BEGIN
    DECLARE v_user_count INT DEFAULT 0;
    DECLARE v_game_count INT DEFAULT 0;
    DECLARE v_already_owned INT DEFAULT 0;

    -- Automated Exception Handler: Roll back on any SQL exception
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transaction Failed: Rolled back completely to maintain Atomicity.' AS
status_message;
    END;

    START TRANSACTION;

    -- Concurrency Check 1: Verify and lock User row
    SELECT COUNT(*) INTO v_user_count FROM `USER` WHERE user_id = p_user_id FOR UPDATE;

    -- Concurrency Check 2: Verify and lock Game row
    SELECT COUNT(*) INTO v_game_count FROM GAME WHERE game_id = p_game_id FOR UPDATE;

    -- Concurrency Check 3: Check if already owned in library
    SELECT COUNT(*) INTO v_already_owned FROM `LIBRARY` WHERE user_id = p_user_id AND game_id =
p_game_id FOR UPDATE;

    IF v_user_count = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Purchase Aborted: User not found.';
    END IF;

    IF v_game_count = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Purchase Aborted: Game not found.';
    END IF;

    IF v_already_owned > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Purchase Aborted: Game already owned in
Library.';
    END IF;

    -- Step 1: Insert Payment
    INSERT INTO PAYMENT (user_id, game_id, amount, payment_status, payment_method, payment_date)
    VALUES (p_user_id, p_game_id, p_amount, 'Completed', p_payment_method, CURDATE());

    -- Step 2: Establish Savepoint
    SAVEPOINT BeforeLibraryUpdate;

    -- Step 3: Insert Library Record
    INSERT INTO `LIBRARY` (user_id, game_id, purchase_date, status)
```

```
    VALUES (p_user_id, p_game_id, CURDATE(), 'Owned');

    COMMIT;

    SELECT 'Purchase completed successfully!' AS status_message,
           p_user_id AS user_id, p_game_id AS game_id, p_amount AS amount_paid;
END$$

DELIMITER ;
```

7. Conclusion

The **GameRig** Digital Game Library Management System demonstrates the end-to-end design, implementation, and rigorous optimization of an enterprise relational database. Through systematic progressive phases:

- **Phase I:** Successfully decomposed business requirements into six normalized entities in Boyce-Codd Normal Form (BCNF), eliminating redundancy and mapping full functional dependencies.
- **Phase II:** Established clean DDL table definitions with domain, entity, and referential constraints, populating baseline records and verifying aggregate analytical queries.
- **Phase III:** Deployed multi-table complex joins, nested and correlated subqueries, abstraction views, stored functions, and stored procedures to streamline business reporting.
- **Phase IV:** Hardened the platform with full ACID transaction handling, `FOR UPDATE` pessimistic row-level locking, automated exception rollback handlers, and rigorous integrity violation testing.

The resulting database delivers a robust, highly reliable foundation capable of scaling to thousands of concurrent gaming transactions while guaranteeing zero data loss, orphan records, or concurrency anomalies.

8. References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. MySQL 8.0 Reference Manual: *InnoDB Locking and Transaction Model*. Oracle Corporation. Retrieved from dev.mysql.com.
4. Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
5. Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377-387.

9. Appendix

Appendix A: System Configuration & Environment

Component	Technology / Version	Configuration & Port
Database Server	MySQL Server 8.0 Community Edition	`localhost:3306` (Default DB: `GameRig`)
Backend Server	Node.js v24.19.0 + Express 4.19.2	`http://localhost:5050` (REST API Engine)
Frontend Client	React 18.2.0 + Vite 5.1.6	`http://localhost:3000` (SPA with API Proxy)
Database Driver	mysql2 (Connection Pool mode)	Promise-based async/await queries

Appendix B: REST API Endpoints & Database Mapping

HTTP Method	Endpoint Route	Target SQL Object / Table	Operation Performed
`GET`	`/api/games`	`GAME` + `REVIEW`	Fetch catalog with calculated average star rating
`POST`	`/api/auth/login`	`USER` / `ADMIN`	Authenticate email and bcrypt-hashed password
`POST`	`/api/library/buy`	`ProcessGamePurchase()`	Execute transaction procedure (Payment + Library)
`GET`	`/api/library/:userId`	`Game_Library_Summary`	Retrieve user-owned library titles
`POST`	`/api/reviews`	`AddGameReview()`	Execute review insertion stored procedure
`GET`	`/api/admin/metrics`	Aggregation Queries	Real-time user count, game count, and gross revenue

Appendix C: Evaluation Credentials

Role	Email Address	Password	Privileges
End User	`john@gamerig.com`	`password123`	Catalog Browse, Buy Games, Personal Library, Submit Reviews
Admin	`admin@gamerig.com`	`password123`	Catalog Management, User Administration, Revenue Auditing